Race Conditions and Synchronization in xv6

Operating Systems Case Study

L Nithish Kumar Goud | 160124733327

G Ishan | 160124733322

Santhosh | 160124733332

Venkat | 160124733340

Faculty / semester / academic year: _____

Problem statement

Concurrent kernel requests update the same physical-page free list.

Unsafe overlapping updates can break the allocator invariant.

Compare original locks, deliberate removal, and restoration.

Question: how does the existing spinlock protect shared state?

Introduction, objectives and motivation

Race condition: correctness depends on concurrent timing.

Objective: connect shared-data rules to actual kernel behavior.

Table 2. Illustrative lost update: expected 2, final value 1

Step	Activity A	Activity B
1	Read 0	Read 0
2	Compute 1	Compute 1
3	Store 1	Store 1

counter++ is a sequence of actions; the example is conceptual.

Existing system and race problems

Standard xv6 already protects its allocator with kmem.lock.

Removing protection permits conflicting free-list updates.

Possible effects: lost pages, corruption, allocation errors, crashes.

A race is a correctness problem; deadlock is a waiting problem.

An observed timeout does not prove deadlock.

Synchronization types and suitable uses

Table 1. Synchronization methods

Method	Suitable purpose
Mutex / spinlock	Exclusive access; sleep or busy-wait as appropriate
Semaphore	Available permits or bounded resources
Condition variable / monitor	Wait for a state predicate under an exclusion rule
Reader-writer lock	Concurrent readers; exclusive writers
Barrier	Wait for all participants at a phase boundary
Atomic operation	Indivisible update; not a whole-object guarantee

Only the xv6 allocator spinlock was evaluated experimentally.

Proposed solution

Retain or restore the same lock around every free-list update.

Protect the complete push or pop, including its dependent reads.

Atomic ownership and acquire/release ordering support the lock.

Keep sections short and follow a consistent lock order.

The solution restores an existing xv6 mechanism.

Synchronization working flow

1. Disable local interrupts with push_off()

2. Acquire ownership through atomic exchange

3. **Read or update the protected free-list state**

4. Release with ordering, then restore nested interrupt state

Local interrupt disabling alone cannot exclude another CPU.

Practical environment and protocol

Table 3. Fixed settings for all three phases

Setting	Recorded configuration
Guest / host	RISC-V xv6 / Ubuntu 24.04 under WSL2
Emulator / compiler	QEMU 8.2.2 / RISC-V GCC 13.3.0
Resources	4 CPUs; 128 MiB RAM
Workload	4 children; 8 pages each; 200 rounds
Attempts	10 per phase; fresh guest per attempt
Limits	30 s to shell; 60 s after stress command

C workload + C host runner; recorded on 24 October 2026.

Shared allocator data and critical sections

```
/* kalloc(): remove one free page */
acquire(&kmem.lock);
r = kmem.freelist;
if (r) kmem.freelist = r->next;
release(&kmem.lock);

/* kfree(): add one free page */
acquire(&kmem.lock);
r->next = kmem.freelist;
kmem.freelist = r;
release(&kmem.lock);
```

Figure 1. Exact protected update excerpts; surrounding code omitted.

C workload and checking logic

Pipe gate releases four child processes to overlap their work.

Each child allocates 32 KiB and writes a distinct byte tag.

Verify every byte, release memory, and repeat 200 rounds.

Parent checks setup errors and every child exit status.

volatile forces rereads here; it does not synchronize the kernel.

Allocation and freeing execution paths

Allocation: sbrk() > sys_sbrk() > growproc() > uvmalloc() > kalloc()

Release: sbrk(-bytes) > growproc() > uvmdealloc() / uvmunmap() > kfree()

```
wsl -d Ubuntu-24.04
cd ~/xv6-member1
make qemu CPUS=4
# In the xv6 shell:
race_test
```

Private user buffers exercise the shared kernel allocator.

Possible allocator interleavings

Table 4. Theoretical example: both CPUs allocate head page P

Step	CPU 0	CPU 1
1	Read head P	
2		Read head P
3	Set head to P.next	
4		Set head to P.next
5	Return P	Return P

Possible source-level schedule; not a trace from a recorded failure.

Original-lock baseline

```
xv6 kernel is booting

hart 1 starting
hart 3 starting
hart 2 starting
init: starting sh
$ race_test
race_test: children=4 pages=8 rounds=200
race_test: PASS (all 4 children completed)
$ QEMU: Terminated
```

Figure 2. Saved C-validation A01 console text; baseline phase: 10/10 PASS.

Deliberate allocator-lock removal

```
/* Same temporary change in kalloc() and kfree(): */

- acquire(&kmem.lock);
+ // acquire(&kmem.lock);

  /* existing free-list updates stay unchanged */

- release(&kmem.lock);
+ // release(&kmem.lock);
```

Figure 3. Focused diff: four calls removed; workload unchanged.

Unlocked-run evidence

: CORRUPTION round=1 offset=16384 expected=3 got=4

race_test: FAIL (2 child failures; setup_failed=0)

Host limit reached: 60 seconds

B06 also contains panic/trap text. Primary categories follow the CSV.

Figure 4. Separate excerpts: B06, B07, B08 respectively; not one transcript.

Lock restoration and verification

kernel/kalloc.c: OK

final-restoration.diff: empty

user/race_test.c: OK

race_test: PASS (all 4 children completed)

Figure 5. Restoration checks and C01 result; restored phase: 10/10 PASS.

Demonstration

Show the workload, controlled lock removal, and restoration.

Open the accompanying walkthrough video.

Earlier-run source/log replay; separate from C-validation results.

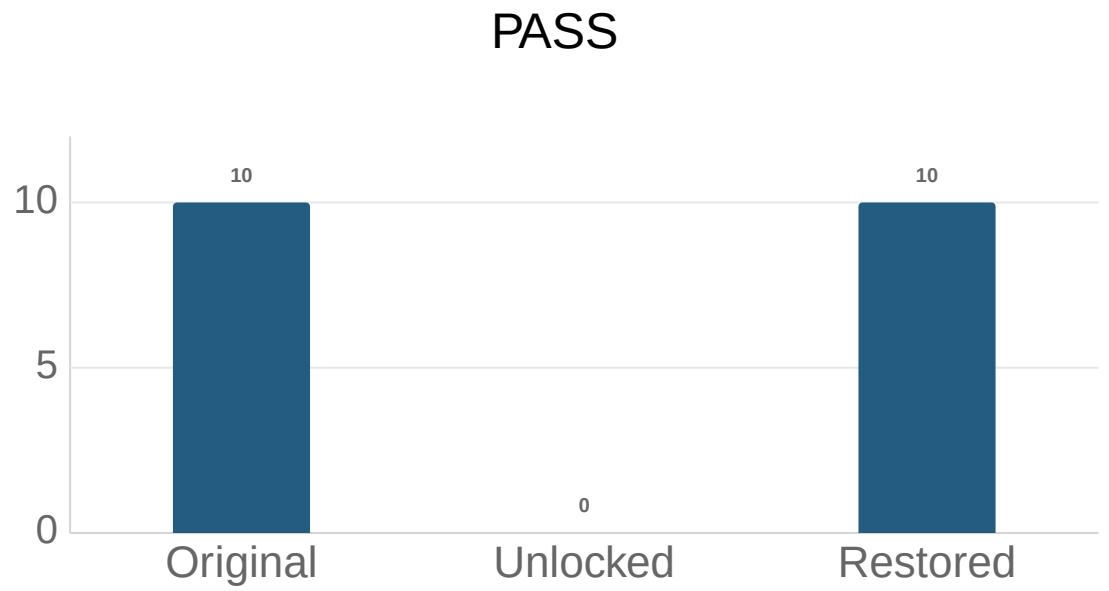
Open demo-walkthrough.mp4

Rendered walkthrough, not a live desktop screen recording.

Results and practical limitations

Table 5. Recorded attempt totals

Phase	PASS	Other
Original	10	0
Unlocked	0	10
Restored	10	0



Unlocked: 1 corruption, 6 panic/trap, 2 test failures, 1 timeout.

Timeout is not a deadlock diagnosis; elapsed times are not benchmarks.

One workload, 4 CPUs, 10 attempts per phase; PASS is not a proof of safety.

Conclusion, applications and future scope

Learning: protect the invariant across the complete critical section.

Finding: completion returned after the original locks were restored.

Applications: memory lists, shared queues, caches, application state.

Future work: vary CPUs, detect lost pages, measure contention.

Further experiments and alternative primitives remain untested.

References

Fixed xv6 source revision: f780d7850765d74cbdcd1fff4b822d8829f7c9bd

C-validation: results.csv, raw logs, diffs, and restoration checks

MIT xv6 book: Locking chapter

POSIX / Linux man-pages: conditions, semaphores, RW locks, barriers

Faculty format: 1000097330.pdf

Full source links and relevant evidence filenames are in speaker notes.